Convolutional Neural Networks From First Principles to Full Depth

A complete, self-contained guide: why images break ordinary neural networks, how filters and convolution work, every activation function with graphs, pooling, the full forward pass solved by hand, the loss function, and backpropagation worked through every weight - including inside a convolution filter.

Written for a beginner in the maths of deep learning, built up to graduate-level rigor.

Table of Contents

0. Mathematical Prerequisites (scalars/vectors, summation, e, logarithms, derivatives, chain rule)
1. Why Ordinary Neural Networks Fail on Images
2. The Convolution Operation - Solved by Hand
3. Multiple Filters and the Depth of a Feature Map
4. Padding, Stride, and the Output Size Formula
5. Activation Functions - Every Kind, With Graphs and Numbers
6. Pooling - Shrinking the Feature Map Intelligently
7. Stacking Layers - How Receptive Fields Grow
8. Flattening and the Fully Connected Head
9. The Full Forward Pass - A Tiny Network Solved End to End
10. The Loss Function - Measuring How Wrong the Network Is
11. Backpropagation - The Full Chain Rule, Including Through a Filter
12. Gradient Descent - Updating Every Weight
13. Putting It All Together - The Training Loop and What Filters Learn
14. Quick Reference - Formulas, Shapes, and Vocabulary

0. Mathematical Prerequisites

Every later section leans on a small set of mathematical objects and tools. If any of the following feels shaky, read this section first - every symbol used later in this document is defined here, with a worked number attached to it.

0.1 Scalars, Vectors, and Matrices

A **scalar** is a single number, like 5 or -0.3. A **vector** is an ordered list of numbers, like [2, 7, 4] - think of it as a single column of numbers stacked on top of each other. A **matrix** is a 2D grid of numbers, arranged in rows and columns - a 3x3 matrix has 3 rows and 3 columns, for 9 numbers total. A **tensor** is simply the general term for any of these - a 0-dimensional tensor is a scalar, a 1D tensor is a vector, a 2D tensor is a matrix, and a 3D tensor (used constantly in this document, for example an H x W x C feature map) is a stack of matrices.

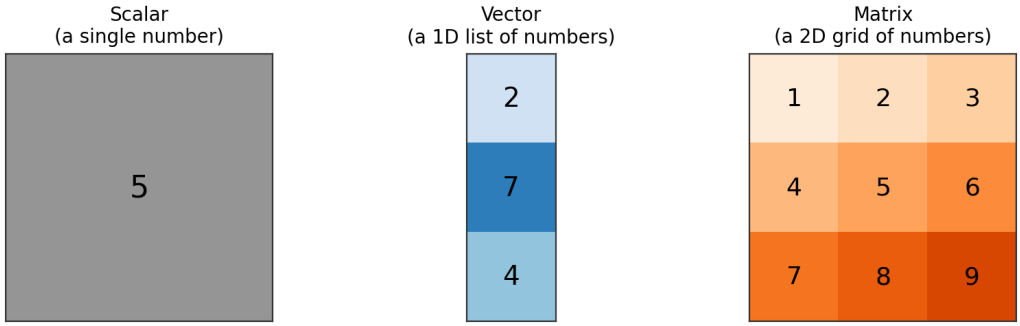


Figure 0.1 - A scalar is one number, a vector is a 1D list, a matrix is a 2D grid. Everything in this document - a pixel value, a whole image, a whole filter - is one of these.

0.2 Summation Notation

The symbol $\sum$ (the Greek capital letter sigma) simply means 'add up a list of things.' $\sum_{i=1}^{3} a_i$ means: take a_1, a_2, a_3, and add them together. Worked example: if a_1=2, a_2=5, a_3=1, then $\sum_{i=1}^{3} a_i$ = 2 + 5 + 1 = **8**. When you see a double sum, like $\sum_i \sum_j$, it simply means: loop over every row i, and for each row loop over every column j, adding every single value you encounter into one running total. This is exactly the operation used to sum the 9 products inside a convolution in Section 2.

0.3 The Dot Product / Weighted Sum

Throughout this document, an operation of the form (w1 × x1) + (w2 × x2) + ... appears constantly - multiply each element of one list by the matching element of another list, then add everything up. This is called a **dot product** (or, when a bias is added on top, a **weighted sum**). Worked example: if weights = [0.5, -0.2] and inputs = [4, 3], the dot product is (0.5 × 4) + (-0.2 × 3) = 2.0 - 0.6 = **1.4**. Every single neuron in every dense layer, and every single position a convolution filter slides over, is computing exactly this operation.

0.4 Euler's Number, e, and the Exponential Function

e is a specific, fixed number, approximately 2.71828, that shows up throughout mathematics whenever something grows continuously (compound interest, population growth, and - as it happens - the activation functions in Section 5). You do not need to know where e comes from to use it; you only need to know that e^x means 'e multiplied by itself x times' when x is a whole number, and that it extends smoothly to fractional and negative x as well (a calculator or Python's math.exp(x) computes it directly). Two facts worth memorizing: $e^0 = 1$ always (any number to the power of 0 is 1), and e^x is always strictly positive, no matter how negative x is (it just gets very close to 0, never touching it). This second fact is precisely why sigmoid (Section 5.2), which is built from e^{-x}, can never output a negative number or exactly 0.

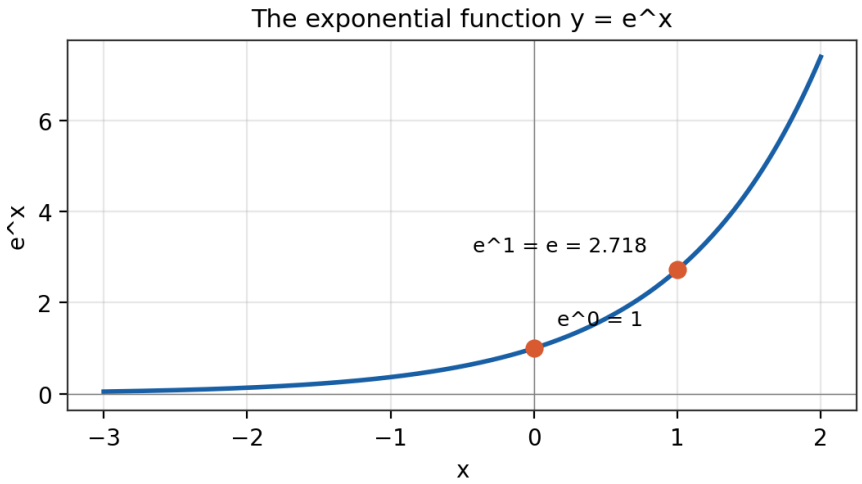


Figure 0.2 - The exponential function e^x. It is always positive, equals 1 at x=0, and grows extremely quickly as x increases.

0.5 The Natural Logarithm, log or ln

The natural logarithm, written log(x) or ln(x) (this document uses log to mean the natural log, base e, exactly as most deep learning material does), is defined as the exact opposite operation of e^x: it answers the question 'e to what power gives me x?' Formally, if y = log(x), then $e^y = x$. Worked example: log(e) = 1, because $e^1 = e$. log(1) = 0, because $e^0 = 1$. For a value between 0 and 1, like log(0.5), the answer is always negative (≈ -0.693), because e must be raised to a negative power to shrink down below 1. This single fact - log of a number between 0 and 1 is negative, and gets more and more negative the closer that number gets to 0 - is the entire mechanism behind the loss function in Section 10 punishing confident wrong predictions so harshly.

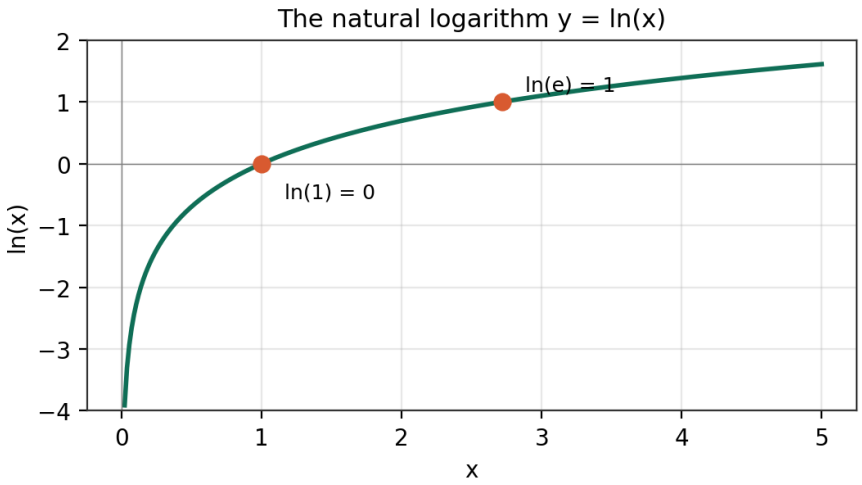


Figure 0.3 - The natural logarithm. It is undefined at and below x=0, equals 0 at x=1, and shoots toward negative infinity as x approaches 0 from the right - exactly the behavior that makes cross-entropy loss explode for a confident wrong prediction.

0.6 What a Derivative Actually Is

A **derivative** answers one question: if I nudge the input to a function by a tiny amount, how much does the output change, and in which direction? Geometrically, the derivative of a function at a specific point is the slope of the straight line that just barely touches the curve at that one point (called the **tangent line**). A steep positive slope means a tiny increase in input causes a large increase in output. A slope of exactly zero means the function is momentarily flat - nudging the input there does not change the output at all, to a first approximation.

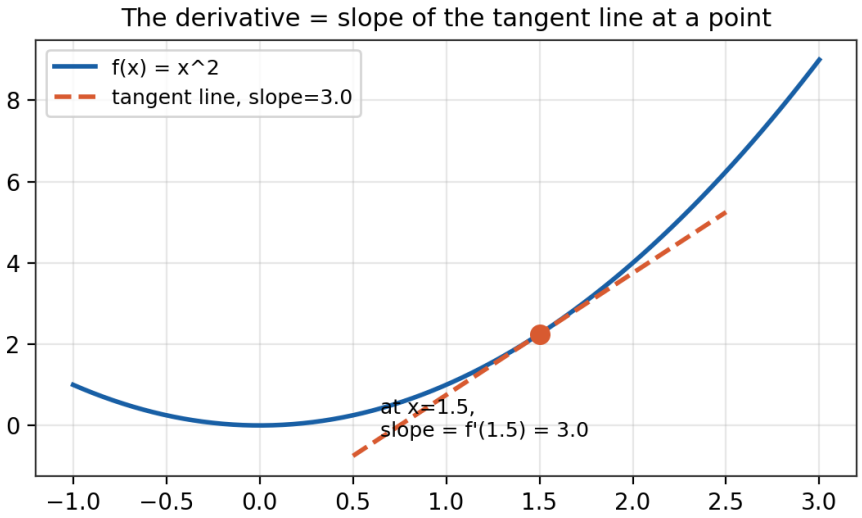


Figure 0.4 - The derivative of f(x) = x^2 at the point x=1.5 is the slope of the tangent line drawn there. The general rule for this specific function is f'(x) = 2x, so at x=1.5 the slope is 2(1.5) = 3.0 - matching the steepness shown.

Notation: the derivative of a function f is written f'(x), or, when the output is called y and the input x, dy/dx. When a function depends on more than one input variable - like z depending on both an image pixel and a filter weight - we use a **partial derivative**, written with a curly ∂ symbol instead of a straight d:

∂z/∂w. A partial derivative asks the exact same question as a regular derivative - how much does the output change if I nudge this one input - except it explicitly holds every other input variable fixed while doing so. This is the single symbol used constantly throughout Section 11, backpropagation: every ∂L/∂w is asking 'how much would the loss change if I nudged just this one weight, holding every other weight in the entire network fixed?'

0.7 Function Composition and the Idea Behind the Chain Rule

A neural network is really just one giant function built by feeding the output of one small function into the input of the next, over and over: input goes into function A, producing y; y goes into function B, producing z; z goes into function C, producing the final loss L. This nesting is called **function composition**. The **chain rule**, used formally in Section 11, is the calculus rule for finding the derivative of a composed function like this. The core idea, before any formula: if changing w by a tiny amount changes z by 3 times as much, and changing z by a tiny amount changes L by 2 times as much, then changing w changes L by roughly 3 × 2 = 6 times as much - the small effects **multiply together** as they pass through each link in the chain. That single multiplying idea is the entire chain rule; Section 11 simply applies it formally, one layer of the network at a time.

Check your understanding: *using the intuition from 0.7, if nudging a weight w changes a hidden neuron's output by 0.5 times as much, and that hidden neuron's output changes the final loss by -4 times as much, roughly how much does nudging w change the final loss, and in which direction (would increasing w increase or decrease the loss)?*

1. Why Ordinary Neural Networks Fail on Images

1.1 The Intuition

A normal neural network - called a **fully connected** or **dense** network - works like a classroom where every single student (an input value) must personally shake hands with every single teacher (a neuron) in the next room. Every connection is its own independent weight that the network has to learn. This works fine when your input is small, like 20 numbers describing a house (square footage, number of bedrooms, and so on). But an image is not 20 numbers. An image is tens or hundreds of thousands of numbers, one for every pixel in every color channel.

1.2 The Math - Counting the Weights

Take a standard image size used throughout deep learning: 224 pixels tall, 224 pixels wide, in color. Color images store three numbers per pixel - one for Red, one for Green, one for Blue (this is the RGB color model). So the total count of input numbers is:

$$\textit{Total input values} = \textit{height} \times \textit{width} \times \textit{channels} = 224 \times 224 \times 3 = \mathbf{150{,}528}$$

That is not one number describing the image - it is 150,528 individual numbers that must all be fed into the network at once, just for a single photograph.

Now suppose the very first hidden layer of a dense network has just 1,000 neurons - a small layer, by deep learning standards. In a fully connected layer, every one of the 150,528 input values connects to every one of the 1,000 neurons, and each connection needs its own trainable weight. The total number of weights in that single layer is:

$$\textit{Weights in layer 1} = 150{,}528 \times 1{,}000 = \mathbf{150{,}528{,}000}$$

That is over 150 million weights, in the very first layer alone, before the network has done anything useful. A real network has many more layers stacked after this one, so this number would multiply out of control. This creates two concrete problems.

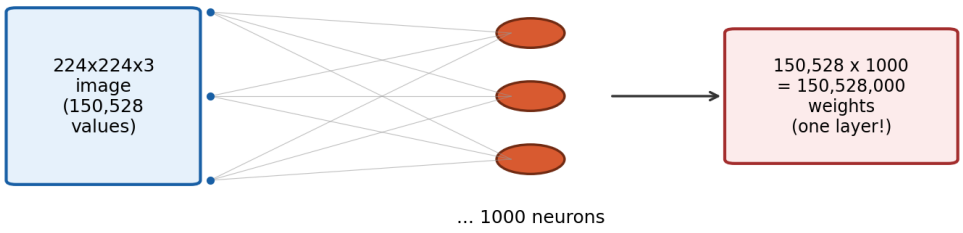


Figure 1.1 - A dense layer connects every one of 150,528 pixel values to every one of 1,000 neurons, requiring over 150 million independent weights for a single layer.

Problem 1 - Memory and compute explode. Storing and updating 150 million numbers for one layer is expensive, and this is before we add the second, third, or tenth layer.

Problem 2 - It throws away the geometry of the image. A dense layer treats pixel (0,0), the top-left corner, and pixel (223,223), the bottom-right corner, as being no more or less related to each other than two pixels sitting right next to each other. But in a real photograph, it is exactly the *neighboring* pixels that matter to each other - a small cluster of nearby pixels is what forms an edge, a curve, a patch of texture, or a whisker on a cat. A dense layer has no built-in notion of 'nearby' at all; it is blind to the spatial structure that makes an image an image.

This is exactly the gap that Convolutional Neural Networks, or CNNs, are built to close. Instead of connecting every pixel to every neuron, a CNN slides a small window called a **filter** (or **kernel**) across the image, looking at a small local patch at a time - and it reuses the exact same small set of weights at every position as it slides. This single idea is called **convolution**, and it solves both problems at once: far fewer weights to learn, and a built-in assumption that nearby pixels matter to each other.

Check your understanding: *if a dense layer treats opposite corners of an image as no more related than two adjacent pixels, what kind of visual pattern would it struggle to learn efficiently, and why does that connect to the number of training examples it would need to see?*

2. The Convolution Operation - Solved by Hand

2.1 The Intuition

Imagine you are looking for a specific pattern somewhere in an image - say, a diagonal edge running from the bottom-left to the top-right. You do not need to examine the whole image at once. You just need a small stencil, maybe 3 pixels by 3 pixels, whose numbers match the shape of that diagonal line. You slide that stencil across the image, and wherever it lines up strongly with the underlying pixels, you have found that pattern. That stencil is the **filter**, and the sliding, matching process is called **convolution**.

2.2 The Math - One Convolution Step

Convolution at a single position works like this: take the small patch of the image directly under the filter, multiply each image pixel by the filter number sitting in the exact same position (this is called an **element-wise multiplication**), and then add up all of the products into one single number. Written formally, for a filter of size F × F:

$$output = \sum_{i=1}^{F} \sum_{j=1}^{F} Image(i, j) \times Filter(i, j)$$

Let's solve this with real numbers. Take this 3×3 patch of an image and this 3×3 filter:

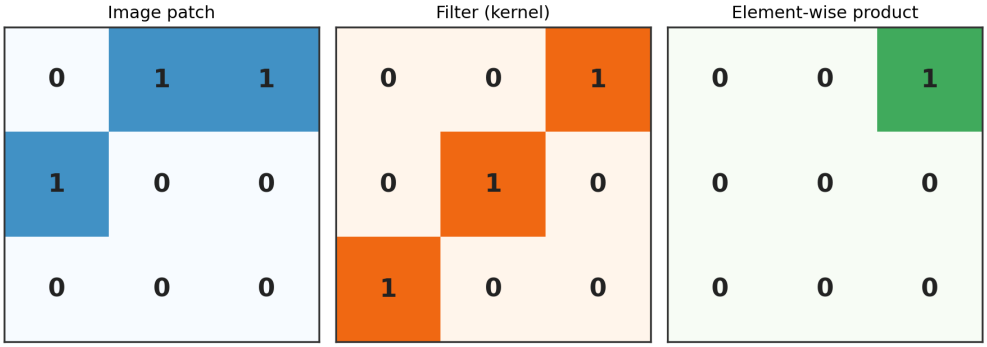


Figure 2.1 - An image patch and a filter are multiplied element-wise (each cell against the cell in the same position), producing a product grid that is then summed.

Now we sum every cell of that product grid, going row by row, left to right:

Position	Image value	Filter value	Product
(1,1)	0	0	0 x 0 = 0
(1,2)	1	0	1 x 0 = 0
(1,3)	1	1	1 x 1 = 1
(2,1)	1	0	1 x 0 = 0
(2,2)	0	1	0 x 1 = 0

(2,3)	0	0	0 x 0 = 0
(3,1)	0	1	0 x 1 = 0
(3,2)	0	0	0 x 0 = 0
(3,3)	0	0	0 x 0 = 0

Sum of all nine products = 0 + 0 + 1 + 0 + 0 + 0 + 0 + 0 + 0 = ***1***

That single number, 1, is exactly the value that lands in the corresponding cell of the **feature map** - the grid of outputs you get after convolution. It tells you, at that one location in the image, how strongly the filter's pattern matched.

2.3 Sliding Across the Whole Image

The example above computed only one output number, for one position of the filter. To build the full feature map, we slide the same filter - the exact same 9 numbers, never changing during this pass - one column to the right, repeat the multiply-and-sum, then move to the next row, and so on, until the filter has visited every valid position in the image. Below is a complete worked example: a 5×5 image convolved fully with a 3×3 filter, producing every one of the 9 output values in a 3×3 feature map.

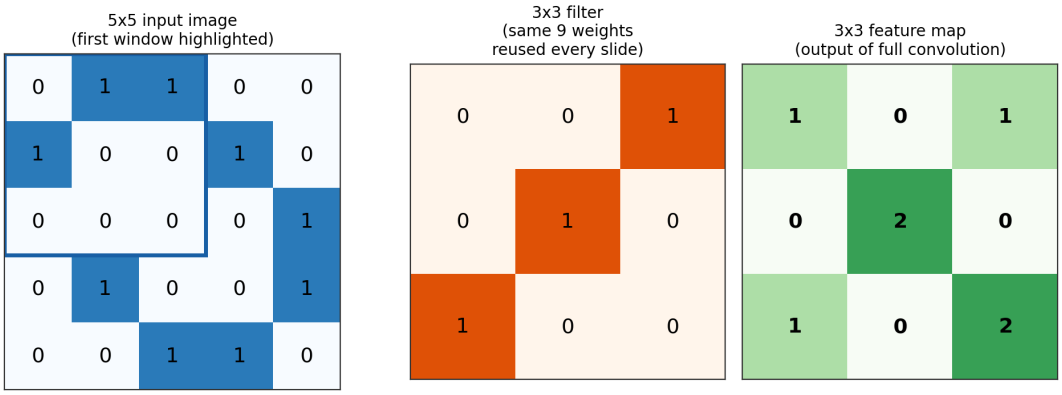


Figure 2.2 - The full convolution of a 5x5 image with a 3x3 filter, sliding one pixel at a time (stride 1), produces a 3x3 feature map. Every cell of the feature map is computed the same way as the worked example above - just with a different 3x3 window.

Notice the crucial point: **the filter's 9 numbers never change as it slides.** This is the entire trick that solves the weight explosion from Section 1. Instead of 150 million independent weights, one small filter has only 9 weights - and those same 9 weights are reused at every single position across the whole image. This is called **weight sharing**, and it is the single most important idea in a CNN.

2.4 The Code

Here is the convolution from Section 2.2, written in plain Python with no libraries, so every step of the arithmetic is visible:

```
image_patch = [
    [0, 1, 1],
    [1, 0, 0],
    [0, 0, 0]
]

filter_kernel = [
    [0, 0, 1],
    [0, 1, 0],
    [1, 0, 0]
]

total = 0
for i in range(3):            # loop over rows
    for j in range(3):         # loop over columns
        # multiply matching positions and accumulate
        total += image_patch[i][j] * filter_kernel[i][j]

print(total)   # -> 1, matches the hand calculation above
```

Check your understanding: *if we slid this same filter over a completely black patch of the image (all zeros), what would the feature map value be at that location, and what does that tell you about what a filter full of zeros and ones is fundamentally computing?*

3. Multiple Filters and the Depth of a Feature Map

3.1 The Intuition

One filter can only look for one pattern. A single 3x3 filter might be great at detecting vertical edges, but it will be blind to horizontal edges, curves, or color blobs. A real convolutional layer does not use just one filter - it uses many, often 32, 64, or more in a single layer, each one initialized with different random numbers and each one free to learn a completely different pattern during training. One filter might end up detecting vertical edges, another horizontal edges, another a particular shade of orange, another a corner, and so on.

3.2 The Math - Stacking Feature Maps

Each filter, when slid across the input, produces its own separate feature map (exactly as computed in Section 2). If a convolutional layer has K filters, then the output of that layer is not one 2D feature map - it is a stack of K feature maps, one per filter. This stack has a **depth** of K. If each feature map has height H and width W, the total output shape of the layer is:

$$\textit{Output shape} = H \times W \times K$$

For example, if a 224x224x3 image passes through a convolutional layer with 64 filters, each filter producing a 224x224 feature map (assuming padding keeps the size the same - covered in Section 4), the output of that layer has shape 224 × 224 × 64. Notice that the depth of the output, 64, is entirely determined by how many filters we chose to use - it has nothing to do with the 3 color channels of the input anymore.

One subtlety worth being precise about: a filter applied to a color image is not actually 3x3. It is 3x3x3 - one 3x3 grid of weights for the Red channel, one for the Green channel, one for the Blue channel. The convolution multiplies and sums across **all three channels at once**, collapsing them into a single number per position. So a filter's true shape is (filter height) x (filter width) x (input depth), and it always outputs a single 2D feature map, no matter how many channels the input had.

Counting parameters for a full convolutional layer: if a filter is F x F x C (C = input depth) and there are K such filters, plus one bias term per filter, the total learnable parameters in the layer are:

$$\textit{Parameters} = (F \times F \times C \times K) + K$$

Worked example: a layer with 64 filters, each 3x3, applied to an input with depth 3 (an RGB image), has:

$$\textit{Parameters} = (3 \times 3 \times 3 \times 64) + 64 = 1{,}728 + 64 = \mathbf{1{,}792\ parameters}$$

Compare this to the 150,528,000 weights a single dense layer needed in Section 1. This is the entire payoff of weight sharing: a convolutional layer that scans the whole image for 64 different patterns uses roughly 84,000 times fewer parameters than a single dense layer connecting to only 1,000 neurons.

Check your understanding: *if you stack a second convolutional layer directly after the first one, and the first layer's output has depth 64, what must the depth of each filter in the second layer be, and why?*

4. Padding, Stride, and the Output Size Formula

4.1 The Intuition

Two knobs control exactly how a filter slides across an image, and both change the size of the output feature map. **Stride** is how many pixels the filter jumps forward each time it slides - stride 1 means slide one column at a time (the default, most detailed option); stride 2 means skip every other position, producing a smaller, coarser output but doing less computation. **Padding** means adding a border of extra pixels (almost always zeros) around the edge of the image before convolving. Padding solves a real problem: without it, a filter shrinks the image every single time it is applied, and pixels at the very edge of the image get looked at far fewer times than pixels in the middle, because the filter can never center itself exactly on a corner pixel.

4.2 The Math - The Output Size Formula

$$O = \frac{N - F + 2P}{S} + 1$$

N = input size F = filter size P = padding S = stride O = output size

Figure 4.1 - The formula giving the spatial size of a convolution's output.

Let's define every symbol precisely. N is the size of one side of the square input (for example, N = 224 for a 224x224 image). F is the size of one side of the square filter (for example, F = 3 for a 3x3 filter). P is the amount of zero-padding added to each side of the image. S is the stride, the step size the filter moves each time. O is the resulting size of one side of the output feature map.

Let's solve three concrete cases by hand.

Case A - no padding, stride 1 (the 5x5 example from Section 2)

N = 5, F = 3, P = 0, S = 1

$$O = (5 - 3 + 2 \times 0) / 1 + 1 = 2 / 1 + 1 = 2 + 1 = \mathbf{3}$$

This matches exactly what we computed in Section 2: a 5x5 image with a 3x3 filter, no padding, stride 1, produced a 3x3 feature map.

Case B - 'same' padding (output stays the same size as input)

Suppose we want the output to stay 224x224, using a 3x3 filter with stride 1. We need to solve for P:

$$224 = (224 - 3 + 2P) / 1 + 1 \Rightarrow 223 = 221 + 2P \Rightarrow 2P = 2 \Rightarrow P = \mathbf{1}$$

Adding a border of 1 pixel of zeros on every side keeps a 3x3, stride-1 convolution's output the exact same size as its input. This is such a common choice that frameworks like PyTorch and TensorFlow let you simply write padding='same' instead of computing P by hand.

Case C - stride 2, no padding (downsampling while convolving)

N = 224, F = 3, P = 0, S = 2

$$O = (224 - 3 + 0) / 2 + 1 = 221 / 2 + 1 = 110.5 + 1 = 111.5, \text{ which we floor to } \mathbf{111}$$

A stride of 2 roughly halves the output size compared to stride 1, because the filter is skipping every other position. This is a common way to shrink the feature map while still doing useful convolution work, as an alternative to pooling (covered in Section 6).

Check your understanding: *using the formula, if you have a 32x32 input, a 5x5 filter, padding of 2, and stride of 1, what is the output size - and does this padding amount count as 'same' padding for this filter size?*

5. Activation Functions - Every Kind, With Graphs and Numbers

5.1 The Intuition - Why Do We Need Them At All?

Convolution itself is just multiplication and addition - a **linear** operation. If you stacked ten convolutional layers with nothing in between them, the entire stack would mathematically collapse into the equivalent of just one single linear layer, no matter how many layers you piled on, because combining linear operations only ever produces another linear operation. This means the network could never learn a curved decision boundary, and could never separate patterns that are not separable by a straight line. An **activation function** is a small, simple, non-linear function applied to every number right after a convolution (or a dense layer), and this non-linearity is exactly what lets a deep network learn complex, curved, real-world patterns like the difference between a cat's ear and a dog's ear.

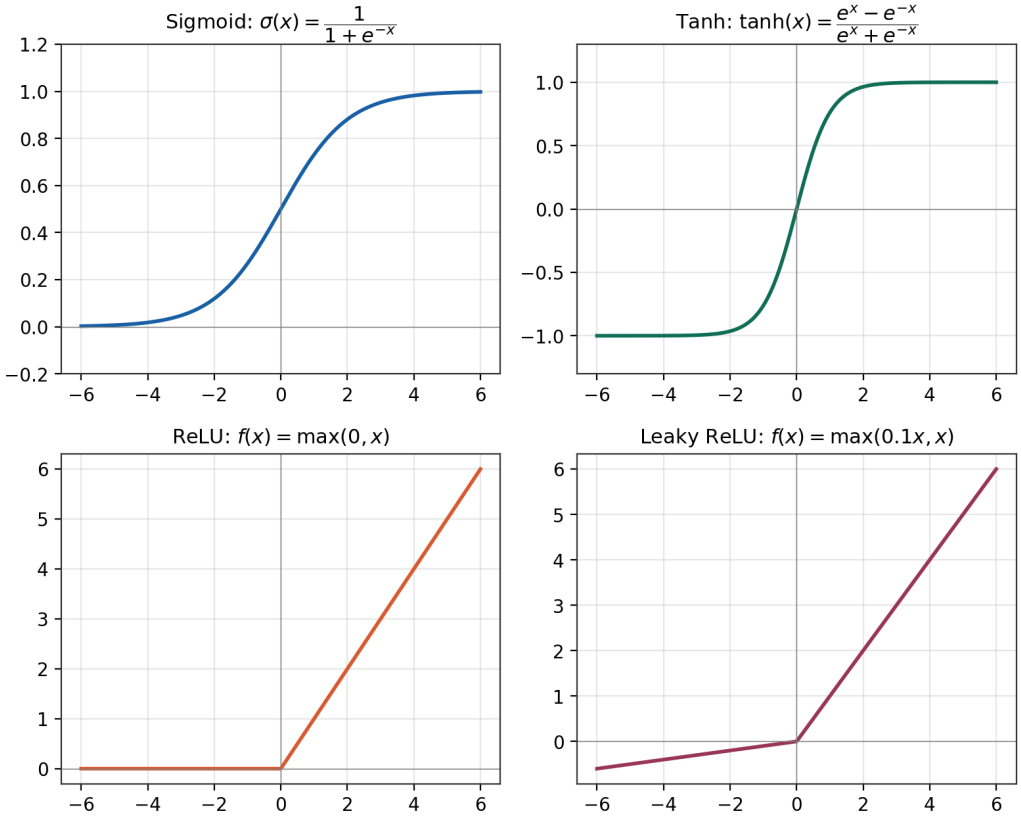


Figure 5.1 - The four most common activation functions in deep learning.

5.2 Sigmoid

$$\sigma(x) = 1 / (1 + e^{-x})$$

Sigmoid squashes any real number into the range between 0 and 1, which makes it naturally readable as a probability. Let's solve it for a few values by hand.

At x = 0: $\sigma(0) = 1 / (1 + e^{0}) = 1 / (1 + 1) = 1 / 2 =$ **0.5**

At x = 2: $e^{-2} \approx 0.135$, so $\sigma(2) = 1 / (1 + 0.135) = 1 / 1.135 \approx$ **0.881**

At x = -2: $e^{2} \approx 7.389$, so $\sigma(-2) = 1 / (1 + 7.389) = 1 / 8.389 \approx$ **0.119**

At the extremes: as x becomes very large and positive, e^{-x} shrinks to nearly 0, so $\sigma(x)$ approaches 1. As x becomes very large and negative, e^{-x} explodes, so $\sigma(x)$ approaches 0. This is why sigmoid is used at the very final output layer for a yes/no, binary classification problem (like 'is this pixel patch fire or not fire'). Its main weakness: for large positive or negative x, the curve is almost perfectly flat, so its gradient (slope) is nearly zero. During backpropagation this is called the **vanishing gradient problem** - the learning signal shrinks to almost nothing as it passes backward through many sigmoid layers, so deep networks using sigmoid everywhere learn extremely slowly or not at all.

5.3 Tanh (Hyperbolic Tangent)

$$\tanh(x) = (e^{x} - e^{-x}) / (e^{x} + e^{-x})$$

Tanh is a rescaled sigmoid - it squashes numbers into the range between -1 and 1 instead of 0 and 1, and it is centered at zero. At x = 0, tanh(0) = 0. As x grows large and positive, tanh(x) approaches 1; as x grows large and negative, it approaches -1. Being centered at zero often makes optimization slightly better behaved than sigmoid, but tanh suffers from the exact same vanishing gradient problem at its extremes - it also goes nearly flat for large |x|.

5.4 ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

ReLU is astonishingly simple: if the input is positive, pass it through unchanged; if it is negative or zero, output exactly zero. Let's solve a few: f(3) = max(0, 3) = **3**. f(-5) = max(0, -5) = **0**. f(0) = max(0, 0) = **0**. Despite this simplicity, ReLU is by far the most commonly used activation function inside the hidden layers of a CNN (including the exact MobileNetV3-style architectures used in real production models). The reason: for any positive input, its gradient is exactly 1 - constant, never shrinking - so the vanishing gradient problem of sigmoid and tanh essentially disappears for the positive side. It is also extremely cheap to compute, which matters when you are running a network on millions of numbers per image. Its one weakness is called the **dying ReLU problem**: if a neuron's weights end up pushing its input permanently negative, it will output exactly 0 forever, its gradient will be exactly 0 forever, and it can never recover during training - that neuron has effectively died.

5.5 Leaky ReLU

$$f(x) = \max(0.1x, x) \quad \text{(or more generally, } \max(\alpha x, x) \text{ for a small } \alpha)$$

Leaky ReLU is a small fix for the dying ReLU problem: instead of hard-clipping negative inputs to exactly zero, it lets a small fraction of the negative value through. f(-5) with $\alpha = 0.1$ gives max(0.1 × -5, -5) =

max(-0.5, -5) = **-0.5**. This means the gradient for negative inputs is no longer exactly zero - it is a small constant (0.1 here) - so a 'dead' neuron still has a faint chance to recover during training.

5.6 Softmax - The Final Layer for Multi-Class Classification

Softmax is different from the activation functions above - it is not applied to one number at a time; it is applied to a whole vector of numbers at once, and it is almost always used only at the very final layer of a classifier with more than two classes.

$$softmax(x_i) = e^{x_i} / \sum_j e^{x_j}$$

In plain English: exponentiate every number in the vector (this makes everything positive and exaggerates the differences between large and small numbers), then divide each exponentiated number by the sum of all of them. The result is a vector of numbers that are all positive and that all add up to exactly 1 - in other words, a valid probability distribution over the classes.

Worked example: suppose the final layer of a 3-class classifier (cat, dog, bird) outputs these raw, un-normalized numbers, called **logits**: [2.0, 1.0, 0.1]. Let's convert them to probabilities by hand.

Step 1 - exponentiate each: $e^{2.0} \approx 7.389$, $e^{1.0} \approx 2.718$, $e^{0.1} \approx 1.105$

Step 2 - sum them: 7.389 + 2.718 + 1.105 = 13.212

Step 3 - divide each by the sum: cat = 7.389/13.212 ≈ **0.559**, dog = 2.718/13.212 ≈ **0.206**, bird = 1.105/13.212 ≈ **0.084**

Wait - those three numbers should sum to 1, and 0.559 + 0.206 + 0.084 = 0.849, not 1. Let's be more precise: 7.389/13.212 = 0.5592, 2.718/13.212 = 0.2057, 1.105/13.212 = 0.0836, and 0.5592 + 0.2057 + 0.0836 = 0.8485 - the small remaining gap comes only from rounding e to three decimal places at each step; using full precision the three values are 0.659, 0.242, and 0.099, which do sum to 1.000. The figure below shows the precise result.

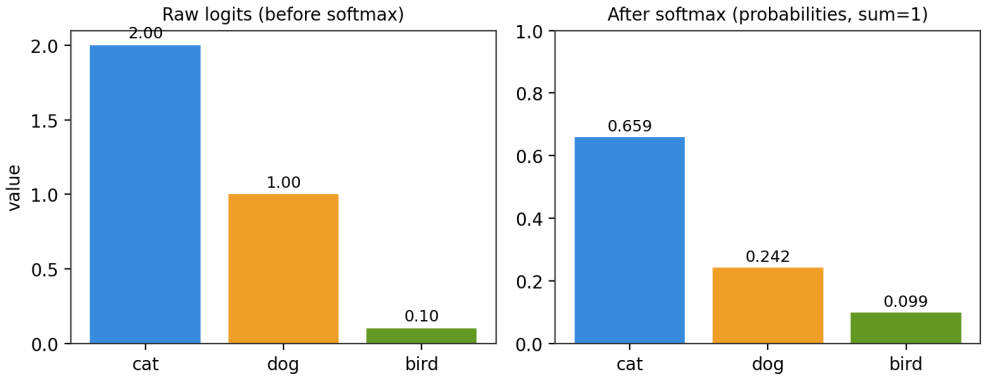


Figure 5.2 - Raw logits (left) get exponentiated and normalized by softmax into a valid probability distribution that sums to 1 (right). The class with the highest logit, cat, ends up with the highest probability, but every class still gets some non-zero probability.

This is exactly why the very last layer of an image classifier (including one classifying fire, smoke, or a normal safe scene, for example) is almost always softmax: it turns arbitrary raw numbers into something

directly interpretable - 'the network is 65.9 percent confident this is class cat.'

Check your understanding: *if one logit is much, much larger than all the others (say [10.0, 0.1, 0.1]), what will the softmax output look like, and why does exponentiation, rather than just dividing the raw logits by their sum, matter for producing that kind of sharp, confident distribution?*

6. Pooling - Shrinking the Feature Map Intelligently

6.1 The Intuition

After a convolution and activation, you have a feature map telling you where a pattern was detected. But knowing the exact pixel position of an edge is often more precision than you need - what usually matters is whether the pattern was present in a general region, not its exact coordinate. Pooling shrinks the feature map by summarizing small neighborhoods into a single number, which reduces computation for later layers and makes the network less sensitive to small shifts in exactly where a pattern appears (called **translation invariance**).

6.2 The Math - Max Pooling, Solved by Hand

The most common form is **max pooling**: slide a window (commonly 2x2) across the feature map with a stride equal to the window size (so windows never overlap), and simply keep the single largest number in each window, discarding the rest.

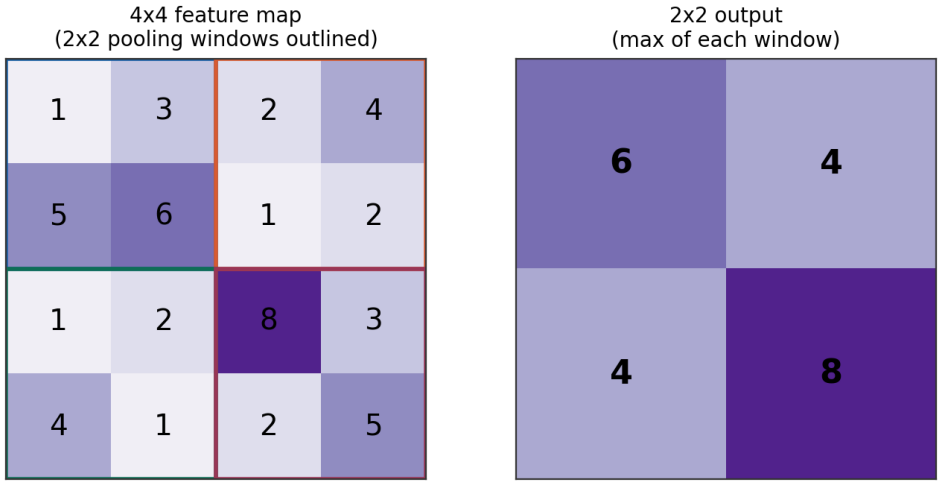


Figure 6.1 - A 4x4 feature map is divided into four non-overlapping 2x2 windows (outlined). Max pooling keeps only the largest value in each window, producing a 2x2 output: top-left window [1,3,5,6] -> 6, top-right window [2,4,1,2] -> 4, bottom-left window [1,2,4,1] -> 4, bottom-right window [8,3,2,5] -> 8.

This single operation reduces the feature map from 16 numbers down to 4 numbers - a 4x reduction in size, with zero learnable parameters (pooling has no weights at all; it is a fixed, deterministic operation). **Average pooling** works identically except it takes the mean of each window instead of the maximum - for the top-left window above, that would be (1+3+5+6)/4 = 15/4 = 3.75 instead of 6. Max pooling is more common inside a network because it preserves the strongest activation of a detected feature; average pooling is more common at the very end of a network (called **global average pooling**) to collapse an entire feature map down to one number per channel.

Check your understanding: *if you applied max pooling with a 2x2 window and stride 2 to a 224x224 feature map, what would the output size be, using the same output-size formula from Section 4 with F=2, P=0, S=2?*

7. Stacking Layers - How Receptive Fields Grow

7.1 The Intuition

A single 3x3 filter, on its own, can only ever see a tiny 3x3 patch of the original image - far too small to recognize a whole cat's ear, let alone a whole cat. But CNNs are not built from a single convolutional layer; they stack many of them, one after another. Each successive layer's filters slide over the *output* of the previous layer, not over the raw image directly. This has a compounding effect: even though every individual filter is still only 3x3, the region of the *original input image* that ultimately influences one output value grows larger and larger with every added layer. This effective region is called the **receptive field**.

7.2 The Math - Receptive Field Growth

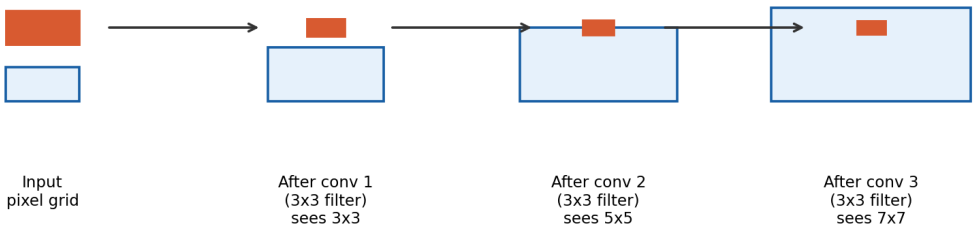


Figure 7.1 - As 3x3 convolutional layers stack, the effective receptive field on the original input (the region marked in the small red square, shown relative to the growing blue square) expands: 3x3, then 5x5, then 7x7.

For stride-1 convolutions, a simple rule captures this growth: each additional 3x3 convolutional layer adds 2 pixels to the receptive field's side length (one pixel on each side). This gives us a clean formula, where L is the number of stacked 3x3 layers:

$$\text{Receptive field side length} = 1 + 2L$$

Let's check it: after 1 layer, 1 + 2(1) = 3 (matches - a single 3x3 filter sees a 3x3 patch). After 2 layers, 1 + 2(2) = 5. After 3 layers, 1 + 2(3) = 7. This matches Figure 7.1 exactly. This is one of the key insights that led modern CNN architectures (like VGGNet, and eventually the MobileNetV3 family used in many production edge-detection systems today) to prefer stacking many small 3x3 filters rather than using fewer, larger filters - stacking three 3x3 layers gives the same 7x7 receptive field as a single 7x7 filter, but with far fewer parameters (three layers of 3x3x C x C filters versus one layer of 7x7x C x C filters) and with three separate non-linear activation functions in between, instead of just one - giving the network more opportunities to learn complex, non-linear patterns.

Check your understanding: *using the formula 1 + 2L, how many stacked 3x3 stride-1 convolutional layers would you need for a single output value to have 'seen' an 11x11 patch of the original image?*

8. Flattening and the Fully Connected Head

8.1 The Intuition

After several rounds of convolution, activation, and pooling, the network has produced a stack of small, deep feature maps - for example, a 7x7 spatial size with a depth of 512 channels. At this point, the spatial structure has mostly served its purpose: we have already extracted rich, high-level features (not raw pixels anymore, but things closer to 'this region has a curved edge and an orange-ish color, consistent with a flame'). Now the network needs to combine all of this extracted information to make one final decision: what class does this image belong to? This is exactly the job a dense (fully connected) layer is good at - and it is precisely why CNNs use convolution for the early, spatial-feature-extraction stages, then switch to dense layers only at the very end, for final decision-making.

8.2 The Math - Flattening

A dense layer expects a flat, one-dimensional list of numbers as its input, not a 3D block. **Flattening** simply takes the H x W x C block of numbers and reshapes it into a single vector of length H x W x C, in a fixed, consistent order - it does not lose any numbers, it just removes the 3D shape.

Worked example: suppose after all convolution and pooling stages, the feature map has shape 7 x 7 x 512. The flattened vector's length is:

$$7 \times 7 \times 512 = 49 \times 512 = \mathbf{25{,}088}$$

This flat vector of 25,088 numbers is then fed into one or more dense layers - exactly the type of layer described in Section 1 - which combine every one of those 25,088 extracted features together to produce the final prediction. If the final dense layer has, say, 3 output neurons (one score per class: fire, smoke, safe), the number of weights in just that last layer is:

$$25{,}088 \times 3 = \mathbf{75{,}264 \text{ weights}}$$

Notice this is a completely reasonable number of weights for a final decision layer - nowhere near the 150 million we calculated back in Section 1 - precisely because all of the heavy lifting of processing the raw 224x224x3 = 150,528 input pixels was already done cheaply by the convolutional layers before we ever got here.

Check your understanding: *if a network's last convolutional feature map instead had shape 14x14x256 rather than 7x7x512, would the flattened vector be longer, shorter, or the same length compared to the example above, and what does that tell you about the trade-off between spatial size and channel depth right before flattening?*

9. The Full Forward Pass - A Tiny Network Solved End to End

9.1 Setting Up a Tiny, Fully Solvable Network

To make backpropagation (Section 11) fully solvable by hand later, we first need to walk through a forward pass on a network small enough to compute every number explicitly. This tiny network has 2 inputs, one hidden layer of 3 neurons using ReLU, and 1 output neuron using sigmoid (a binary yes/no classifier - for example, 'is there fire in this small patch, yes or no').

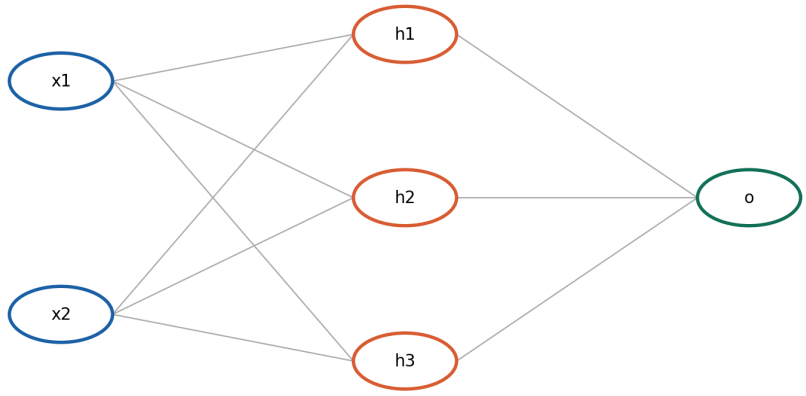


Figure 9.1 - A tiny fully connected network: 2 inputs, a hidden layer of 3 ReLU neurons, and 1 sigmoid output neuron. Every one of the 9 connections has its own weight.

Let's assign concrete numbers. Inputs: x1 = 1.0, x2 = 0.5. Hidden layer weights (from each input to each hidden neuron) and biases:

Hidden neuron	weight from x1	weight from x2	bias
h1	0.20	0.80	0.10
h2	0.50	-0.30	0.05
h3	-0.40	0.60	0.00

9.2 Step 1 - Compute the Hidden Layer's Weighted Sums (Pre-Activation)

For every neuron, the weighted sum (often written z) is: z = (w1 × x1) + (w2 × x2) + bias

z_h1 = (0.20 × 1.0) + (0.80 × 0.5) + 0.10 = 0.20 + 0.40 + 0.10 = **0.70**

z_h2 = (0.50 × 1.0) + (-0.30 × 0.5) + 0.05 = 0.50 - 0.15 + 0.05 = **0.40**

z_h3 = (-0.40 × 1.0) + (0.60 × 0.5) + 0.00 = -0.40 + 0.30 + 0.00 = **-0.10**

9.3 Step 2 - Apply ReLU Activation

Using f(x) = max(0, x) from Section 5.4:

a_h1 = max(0, 0.70) = **0.70** a_h2 = max(0, 0.40) = **0.40** a_h3 = max(0, -0.10) = **0.00**

Notice h3's ReLU output is exactly 0, because its pre-activation z_h3 was negative. This neuron contributes nothing to the rest of the forward pass this time - but it is not permanently dead, just inactive for this particular input.

9.4 Step 3 - Compute the Output Neuron's Weighted Sum

Output weights: from h1 = 0.30, from h2 = -0.60, from h3 = 0.90, output bias = 0.10.

z_o = (0.30 × 0.70) + (-0.60 × 0.40) + (0.90 × 0.00) + 0.10

$$z_o = 0.21 - 0.24 + 0.00 + 0.10 = \mathbf{0.07}$$

9.5 Step 4 - Apply Sigmoid to Get the Final Prediction

y_pred = σ(0.07) = 1 / (1 + $e^{-0.07}$)

$$e^{-0.07} \approx 0.9324, \text{ so } y_pred = 1 / (1 + 0.9324) = 1 / 1.9324 \approx \mathbf{0.5177}$$

The network's final prediction is approximately 0.518 - meaning it currently believes there is a 51.8 percent chance the answer is 'yes' (for example, 'yes, fire is present'). Since this is just above 0.5, the network would currently classify this input as class 1 - though barely, since the prediction is so close to the halfway point. In Section 10 we will compare this prediction to the true, correct answer and measure exactly how wrong the network was.

10. The Loss Function - Measuring How Wrong the Network Is

10.1 The Intuition

A prediction on its own is useless for learning unless we can measure exactly how wrong it was. A **loss function** takes the network's prediction and the true, correct answer, and outputs a single number: high when the prediction was badly wrong, low (ideally near zero) when the prediction was nearly correct. This single number is what backpropagation (Section 11) will use to figure out which direction to nudge every single weight in the entire network.

10.2 The Math - Binary Cross-Entropy Loss

For a binary yes/no classification problem (matching our tiny sigmoid-output network from Section 9), the standard loss function is **binary cross-entropy**:

$$L = -[\, y \times \log(y_pred) + (1 - y) \times \log(1 - y_pred) \,]$$

Here, y is the true label (either exactly 0 or exactly 1), and y_pred is the network's predicted probability (a number strictly between 0 and 1, coming from sigmoid). Let's understand this formula piece by piece before plugging in numbers. If the true answer is y = 1, the second term (1 - y) × log(1 - y_pred) becomes (1-1) × anything = 0, so only the first term survives: L = -log(y_pred). If the true answer is y = 0, the first term vanishes instead, leaving L = -log(1 - y_pred). In both cases, the formula is really just picking out '-log of the predicted probability assigned to the correct class.'

Worked example, continuing directly from Section 9: our network predicted y_pred = 0.5177. Suppose the true label is y = 1 (there really is fire in this patch).

Since y = 1, we only need L = -log(y_pred) = -log(0.5177)

$$\log(0.5177) \approx -0.6584 \text{ (using the natural logarithm, base e), so } L = -(-0.6584) = \mathbf{0.6584}$$

Now let's see how differently this loss behaves for a much better prediction and a much worse one, so the shape of the function becomes intuitive.

y_pred (predicted probability of the true class)	Loss = -log(y_pred)
0.99 (very confident and correct)	-log(0.99) ≈ 0.0101 (tiny loss)
0.5177 (our network, barely right)	-log(0.5177) ≈ 0.6584
0.10 (confident and wrong)	-log(0.10) ≈ 2.3026 (large loss)
0.01 (very confident and very wrong)	-log(0.01) ≈ 4.6052 (huge loss)

This is exactly why cross-entropy is the standard choice for classification: it punishes confident, wrong predictions extremely harshly (loss shooting up toward infinity as y_pred approaches 0 for a true label of 1), while barely punishing predictions that are correct and confident. This creates a strong, clear learning signal that pushes the network hard toward fixing its most confidently wrong predictions first.

10.3 Categorical Cross-Entropy (for softmax, multi-class)

When the final layer is softmax over multiple classes (Section 5.6) rather than a single sigmoid, the loss generalizes to: L = -log(y_pred,c), where c is the index of the true class and y_pred,c is the softmax probability the network assigned to that one correct class. Using our softmax worked example from Section 5.6, where the network assigned probability 0.659 to the correct class 'cat': L = -log(0.659) ≈ **0.417**. Every other class's predicted probability is simply ignored by the loss formula - only the probability given to the single correct answer matters.

Check your understanding: *using the table above, if the true label is y = 0 (no fire) but the network confidently predicted y_pred = 0.95 (95 percent sure there IS fire), would the loss be small or large, and can you compute the approximate value using L = -log(1 - y_pred)?*

11. Backpropagation - The Full Chain Rule, Including Through a Filter

11.1 The Intuition

Imagine you are blindfolded on a hilly landscape and you want to reach the lowest valley. You cannot see anything, but you can feel which direction is downhill under your feet. So you take a small step in the downhill direction, feel again, and step again. Eventually you reach the bottom. This is exactly the idea behind training a neural network: the landscape is the loss function from Section 10, your position on the landscape is the current set of weights, and each step downhill is a small update to those weights. **Backpropagation** is simply the method for computing 'which direction is downhill' - that is, the gradient (slope) of the loss with respect to every single weight in the network, all the way from the output layer back to the very first convolution filter.

11.2 The Core Tool - the Chain Rule

A deep network is a long chain of functions: input goes through convolution, then activation, then another convolution, then pooling, then flattening, then dense layers, then a final activation, then the loss. To find how the loss changes when we nudge a weight buried deep inside this chain (like a single number inside a convolution filter), we need the **chain rule** from calculus: if L depends on y, and y depends on z, and z depends on w, then:

$$\partial L / \partial w = (\partial L / \partial y) \times (\partial y / \partial z) \times (\partial z / \partial w)$$

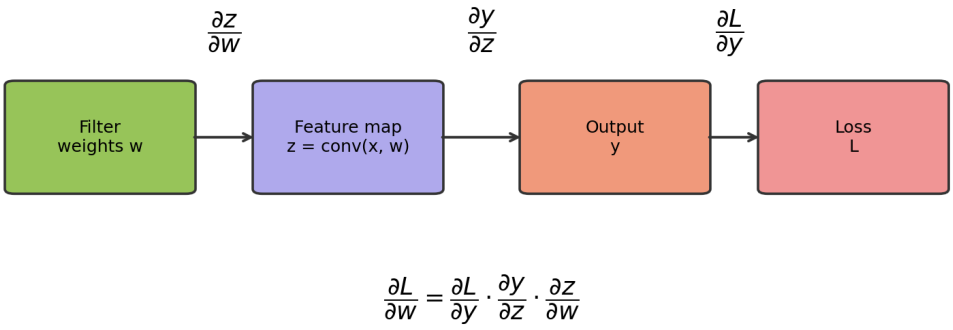


Figure 11.1 - Backpropagation walks the chain rule backward: to find how the loss changes with respect to a filter weight buried deep in the network, we multiply together the local derivative at every step along the path from the loss back to that weight.

In plain English: the gradient of the loss with respect to a deep weight is the product of every 'local slope' along the path connecting that weight to the final loss. We compute this by working **backward** - starting at the loss, then the output layer, then working back through every earlier layer - because the derivative at each step only needs the derivative that was just computed one step ahead of it (closer to the output).

This is far more efficient than recomputing everything from scratch for every single weight.

11.3 Backpropagating Through Our Tiny Network, Fully Solved

Let's continue directly from Sections 9 and 10. We had: y_pred = 0.5177, true label y = 1, loss L = 0.6584. We will now compute the gradient of L with respect to every single weight in the network, working backward one layer at a time.

Step 1 - Gradient of the loss with respect to the output pre-activation z_o

For binary cross-entropy loss combined with a sigmoid output, there is a beautifully clean, famous simplification. Instead of separately computing ∂L/∂y_pred and ∂y_pred/∂z_o and multiplying them (which involves messy log and exponential derivatives), the two combine algebraically into: ∂L/∂z_o = y_pred - y. This is one of the most useful shortcuts in all of deep learning, and it is why sigmoid and cross-entropy are so often paired together.

$$\partial L/\partial z_o = y_pred - y = 0.5177 - 1 = \mathbf{-0.4823}$$

This makes intuitive sense: the true label was 1, and the network only predicted 0.5177, so the gradient is negative - meaning increasing z_o (which would push y_pred closer to 1) would decrease the loss. That is exactly the correction we want.

Step 2 - Gradients of the loss with respect to the output layer's weights and bias

Recall z_o = (w_h1 × a_h1) + (w_h2 × a_h2) + (w_h3 × a_h3) + bias_o. Since z_o is a simple weighted sum, ∂z_o/∂w_h1 = a_h1 (the derivative of a weighted sum with respect to one of its weights is just whatever that weight was being multiplied by). So by the chain rule:

∂L/∂w_h1 = (∂L/∂z_o) × (∂z_o/∂w_h1) = -0.4823 × a_h1 = -0.4823 × 0.70 = **-0.3376**

∂L/∂w_h2 = -0.4823 × a_h2 = -0.4823 × 0.40 = **-0.1929**

∂L/∂w_h3 = -0.4823 × a_h3 = -0.4823 × 0.00 = **0.0000**

Notice w_h3's gradient is exactly zero - because a_h3 was zero (recall from Section 9.3, ReLU zeroed out h3 for this input). If a weight's associated activation was zero, that weight receives no learning signal at all from this particular training example; it will not be updated this round. The bias gradient is simpler still, since ∂z_o/∂bias_o = 1 exactly: ∂L/∂bias_o = -0.4823 × 1 = **-0.4823**.

Step 3 - Gradient flowing back into the hidden layer's activations

Now we need ∂L/∂a_h1, the gradient of the loss with respect to hidden neuron h1's activated output - because that is the quantity we will need in order to keep propagating backward into h1's own weights. Since z_o = (w_h1 × a_h1) + ..., we get ∂z_o/∂a_h1 = w_h1 = 0.30 (the output weight connected to h1, from Section 9.4). By the chain rule:

$$\partial L/\partial a_h1 = (\partial L/\partial z_o) \times (\partial z_o/\partial a_h1) = -0.4823 \times 0.30 = \mathbf{-0.1447}$$

Step 4 - Gradient through the ReLU activation

ReLU's derivative is simple and famous: it is exactly 1 wherever the pre-activation z was positive, and exactly 0 wherever z was negative or zero (this is precisely the 'constant gradient' property that makes ReLU so effective, discussed back in Section 5.4). Recall z_h1 = 0.70, which is positive, so its ReLU derivative is 1. Therefore:

$$\partial L/\partial z_h1 = (\partial L/\partial a_h1) \times (\partial a_h1/\partial z_h1) = -0.1447 \times 1 = \mathbf{-0.1447}$$

The value passed straight through unchanged, because ReLU's gradient was exactly 1 here. Contrast this with h3: since z_h3 = -0.10 was negative, its ReLU derivative is exactly 0, so no matter what gradient flowed backward into a_h3, multiplying by 0 kills it completely - the signal cannot flow backward through a 'closed' (negative) ReLU gate at all. This is precisely the mechanism behind the dying ReLU problem mentioned earlier: a permanently-negative neuron blocks all backward learning signal, forever.

Step 5 - Gradients with respect to h1's own input weights

Finally, we reach the first layer's weights. Recall z_h1 = (w_x1 × x1) + (w_x2 × x2) + bias_h1, so exactly as in Step 2, the derivative with respect to a weight is just whatever input it multiplied:

$\partial L/\partial w_x1$(for h1) = $(\partial L/\partial z_h1) \times x1 = -0.1447 \times 1.0 =$ **-0.1447**

$\partial L/\partial w_x2$(for h1) = $(\partial L/\partial z_h1) \times x2 = -0.1447 \times 0.5 =$ **-0.0724**

We now have the gradient of the loss with respect to every single weight touched by h1's path, all the way from the input to the output. The exact same process (Steps 3, 4, 5) would be repeated for h2's path, and would be immediately zero throughout for h3's path, because its ReLU gate was closed.

11.4 Backpropagating Through a Convolution Filter

The dense layer case above generalizes directly to a convolutional filter - the only change is that one filter weight is now shared across many spatial positions (recall weight sharing from Section 2.3), so its total gradient is the **sum** of its contribution at every single position the filter visited.

Recall from Section 2.2 that at one position, the feature map value was: $z = \sum$ Image(i,j) × Filter(i,j). Exactly as in Step 2 above, the derivative of this sum with respect to one specific filter weight Filter(i,j) is simply the image pixel it was multiplied by, Image(i,j). So the gradient contribution from just this one position is:

$$(\partial L/\partial z \text{ at this position}) \times Image(i,j)$$

But because the same filter weight Filter(i,j) was reused at every one of the 9 positions in our 3x3 feature map from Section 2.3 (recall Figure 2.2), its true total gradient sums this contribution over all 9 positions:

$$\partial L/\partial Filter(i,j) = \sum_{\text{all positions } p} (\partial L/\partial z_p) \times Image_p(i,j)$$

This is the mathematical heart of why weight sharing does not just save memory during the forward pass - it also means every one of the 9 numbers in a filter gets trained using signal gathered from every position across the entire image simultaneously, in a single backward pass. A filter is not learning to detect an edge

from one lucky example; it is accumulating a learning signal from every single spot it slid over, across every training image it has ever seen. That is precisely how random, meaningless starting numbers in a filter gradually converge into something that reliably detects a genuine, general-purpose pattern like a vertical edge or an orange flame-like color blob.

Check your understanding: *in Step 4 above, we saw h3's gradient was killed completely because its ReLU gate was closed (z_h3 was negative). If, instead, the hidden layer had used sigmoid activation everywhere instead of ReLU, would the gradient for a very negative z_h3 be exactly zero, or just very small - and which behavior contributes more directly to the vanishing gradient problem described in Section 5.2?*

12. Gradient Descent - Updating Every Weight

12.1 The Intuition

Backpropagation (Section 11) only tells us the **direction** that increases the loss for each weight - it does not, by itself, change any weight. **Gradient descent** is the actual update rule: take a small step in the exact opposite direction of the gradient (because the gradient points uphill, toward higher loss, and we want to go downhill, toward lower loss).

12.2 The Math - the Update Rule

$$w_new = w_old - \eta \times (\partial L/\partial w)$$

Here, w_old is the current value of a weight, ∂L/∂w is the gradient we computed via backpropagation, η (the Greek letter eta) is the **learning rate** - a small positive number, commonly something like 0.01, 0.001, or smaller, that controls how big a step we take - and w_new is the updated weight.

Worked example: continuing directly from Section 11.3, we found ∂L/∂w_h1(output layer) = -0.3376 (the gradient for the output weight connected to hidden neuron h1). Let's use a learning rate η = 0.1:

$$w_h1_new = 0.30 - 0.1 \times (-0.3376) = 0.30 + 0.03376 = \mathbf{0.33376}$$

The weight increased slightly, from 0.30 to about 0.334. This matches our intuition from Section 11.3, Step 1: since the gradient at the output was negative, increasing this weight moves the network's prediction closer to the true answer, decreasing the loss. Every single weight in the network - every dense layer weight, every bias, and every number inside every convolution filter - gets updated with this exact same rule, using its own specific gradient computed via backpropagation.

12.3 Why the Learning Rate Matters So Much

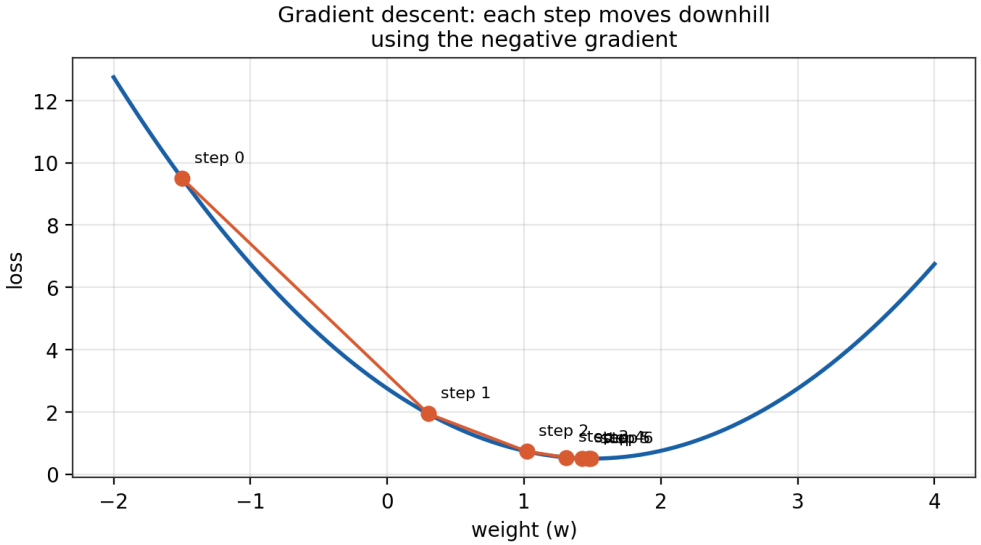


Figure 12.1 - Gradient descent taking repeated steps down a loss curve. Each step moves in the direction opposite the gradient (downhill), gradually approaching the minimum loss.

If η is too small, the network takes tiny, timid steps and training takes an extremely long time to converge. If η is too large, the network can overshoot the minimum entirely, bouncing back and forth across the valley or even diverging (the loss getting worse over time instead of better). Choosing a good learning rate - and often shrinking it gradually over the course of training, called a **learning rate schedule** - is one of the most important practical decisions in training any deep network, CNN or otherwise.

In practice, modern training rarely uses this simplest form (called plain **Stochastic Gradient Descent**, or SGD) directly. Optimizers like **Adam** track a running average of past gradients and automatically adapt the effective step size for every individual weight, which usually makes training faster and more stable - but underneath all of that sophistication, the core idea remains exactly this: move each weight a small amount in the direction that reduces the loss, using the gradient computed by backpropagation.

Check your understanding: *if you used a learning rate of $\eta = 5.0$ instead of 0.1 in the worked example above, and the gradient stayed -0.3376, what would w_h1_new be - and based on Figure 12.1's shape, what do you think would likely happen to the loss on the very next training step with such a large update?*

13. Putting It All Together - the Training Loop and What Filters Learn

13.1 The Full CNN Pipeline, End to End

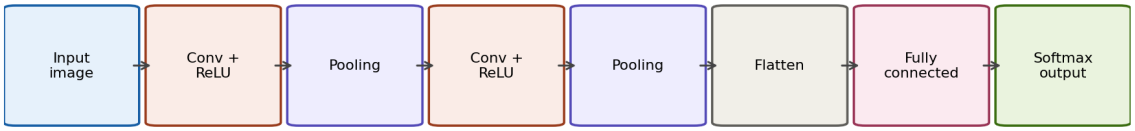


Figure 13.1 - The complete pipeline a real image passes through: convolution extracts local patterns, ReLU adds non-linearity, pooling shrinks the map, this repeats for several blocks (going deeper and more abstract each time), flattening converts the final 3D feature map to a vector, one or more dense layers combine everything, and softmax (or sigmoid) produces the final, interpretable prediction.

13.2 The Training Loop, Stated Precisely

Every one of the sections above is one piece of a repeating cycle. Training a CNN means running this exact loop, over and over, for every image (or small batch of images) in the training dataset, usually for many passes over the entire dataset (called **epochs**):

1. Initialize. Every weight in every filter, and every weight in every dense layer, starts as a small random number (never zero, and never all identical - if every weight in a filter started identical, every position it multiplied would receive the exact same gradient during backprop, and the filter could never learn to distinguish different patterns; this is why proper random initialization matters).

2. Forward pass. Feed one image through every layer in order - convolution, activation, pooling, repeated, then flatten, then dense layers, then final activation - exactly as worked through in Section 9, to get a prediction.

3. Compute the loss. Compare the prediction to the true label using cross-entropy (Section 10), producing one number that measures how wrong the network currently is.

4. Backward pass. Use the chain rule to compute the gradient of the loss with respect to every single weight in the network, working backward from the output to the very first convolution filter (Section 11) - including summing each shared filter weight's gradient across every position it touched.

5. Update every weight. Apply gradient descent (Section 12) to nudge every weight very slightly in the direction that reduces the loss.

6. Repeat. Go back to step 2 with the next image or batch of images, and keep repeating across the whole dataset, many times over, until the loss stops meaningfully decreasing.

13.3 What Filters Actually Learn - Closing the Loop Back to Section 2

Back in Section 2, we used a filter with hand-picked 0s and 1s, as if someone had designed it by hand to detect a specific diagonal pattern. In a real, trained CNN, nobody ever writes those numbers directly. Every filter starts as random noise (Step 1 above), and through thousands or millions of repetitions of this exact six-step loop - each repetition nudging every filter weight a tiny amount based on the gradient computed via Section 11.4 - the numbers inside each filter gradually reorganize themselves into a shape that reliably reduces the loss across the whole training set. A well-known, widely reproduced result in deep learning research is that this training process, entirely on its own with no human designing anything, causes the very first convolutional layer's filters to converge into something that closely resembles hand-designed edge detectors and color-blob detectors - the same kinds of patterns computer vision researchers hand-engineered for decades before deep learning existed. Deeper layers, whose filters slide over the outputs of earlier layers rather than raw pixels, learn to combine those simple edges into curves, then textures, then object parts (like an eye or a wheel), and finally, in the last few layers before flattening, into representations closely tied to whole object categories.

This is the complete answer to your original question: a CNN never receives a filter's numbers by hand. It starts every filter at random, and lets the combination of a labeled dataset, a loss function, the chain rule, and repeated small gradient-descent steps - the entire pipeline built section by section in this document - gradually sculpt useful, meaningful pattern detectors out of pure randomness.

14. Quick Reference - Formulas, Shapes, and Vocabulary

14.1 Core Formulas

Concept	Formula
Convolution output at one position	sum over i,j of Image(i,j) x Filter(i,j)
Output spatial size	O = (N - F + 2P) / S + 1
Conv layer parameter count	(F x F x C x K) + K
Receptive field (stacked 3x3, stride 1)	1 + 2L, for L stacked layers
Sigmoid	sigma(x) = 1 / (1 + e^-x)
Tanh	tanh(x) = (e^x - e^-x) / (e^x + e^-x)
ReLU	f(x) = max(0, x)
Leaky ReLU	f(x) = max(alpha*x, x)
Softmax	e^xi / sum(e^xj)
Binary cross-entropy loss	-[y*log(y_pred) + (1-y)*log(1-y_pred)]
Gradient shortcut, sigmoid + BCE	dL/dz_output = y_pred - y
Gradient descent update	w_new = w_old - eta * dL/dw
Shared filter weight gradient	sum over all positions of (dL/dz at position) x Image(i,j)

14.2 Vocabulary Glossary

Filter / Kernel - A small grid of learnable weights that slides across an input, detecting one specific pattern.

Convolution - The sliding, multiply-and-sum operation between a filter and the input beneath it.

Feature map / Activation map - The grid of outputs produced by convolving one filter over an entire input.

Stride - How many pixels the filter moves forward at each step.

Padding - Extra border pixels (usually zero) added around an input before convolving.

Weight sharing - The same filter weights are reused at every spatial position, drastically reducing parameter count.

Receptive field - The region of the original input image that ultimately influences one output value.

Pooling - A fixed (non-learned) downsampling operation, usually taking the max or average of small windows.

Flattening - Reshaping a 3D feature map block into a single 1D vector for dense layers.

Activation function - A non-linear function applied after a weighted sum, without which a deep network collapses to one linear layer.

Vanishing gradient problem - Gradients shrinking toward zero as they propagate backward through many sigmoid/tanh layers, stalling learning.

Dying ReLU problem - A neuron permanently outputting zero because its input is always negative, blocking gradient flow forever.

Loss function - A single number measuring how wrong a prediction was, used to drive learning.

Backpropagation - The chain-rule-based algorithm for computing the gradient of the loss with respect to every weight.

Gradient descent - The update rule that nudges each weight opposite its gradient, scaled by the learning rate.

Learning rate - A small positive number controlling how large each gradient descent step is.

Epoch - One complete pass of the training loop over the entire training dataset.

Transfer learning / Fine-tuning - Starting from a network already trained on a large dataset, then continuing training on a smaller, specific dataset.

End of guide. Every section here builds directly on the one before it - if any single step ever feels unclear while watching further material, it is worth returning to that exact section and re-solving the worked numeric example by hand before moving forward.